Mauricio Salinas

Back-end Developer · Python · FastAPI · Event-Driven Microservices · Clean Architecture

Santiago, Chile · maundrex@gmail.com · +56 9 4758 6414
mauridev94.github.io · github.com/MauriDev94 · linkedin.com/in/mauricio-salinas-rubio · **English C2 (EF SET 71/100)** · Open to remote work

SUMMARY

Backend developer focused on **Python**, **REST APIs**, and **event-driven microservices**. I build services with **FastAPI** and **Django/DRF** applying **SOLID, Clean Architecture, DDD, and TDD** — spec first, implementation backed by real coverage: **274 tests at 98.7% branch coverage** in my main project and **138 tests** in the microservices system, with integration run via **testcontainers**. Experienced in **PostgreSQL**, **messaging with RabbitMQ** (idempotency, dead-letter queues, eventual consistency), advanced authentication (JWT/OAuth2/OTP/SSO), **observability** (JSON logging + correlation IDs), and **CI/CD pipelines with GitHub Actions**. English C2 for technical documentation and remote teamwork.

EXPERIENCE

Backend Development — Personal Projects

Santiago, Chile · Apr 2025 – Present

- Designed and built **production-grade REST APIs** with FastAPI and Django/DRF applying Clean Architecture, SOLID, DDD, and TDD.
- Designed **event-driven distributed systems** with RabbitMQ: idempotency, dead-letter queues, retries with backoff, and eventual consistency across services.
- Built **CI/CD** pipelines with GitHub Actions (lint, type-checking, tests against real PostgreSQL, enforced coverage) and continuous deployment to Render.

Support Analyst

Cadaques (Domino) · Santiago, Chile · Apr 2023 – Jul 2024

- **Analyzed and resolved incidents** in production systems, performing root-cause diagnosis.
- Queried and maintained **SQL** databases in production; strategic reporting for the operations team.
- Built **Python scripts** to automate operational processes, reducing repetitive manual work.
- Coordinated with the development team on defect escalation and follow-up through resolution.
- Monitored and configured enterprise network infrastructure.

PROJECTS

Event-Driven Orders — Microservices with RabbitMQ

> **`3 microservices · 138 tests · 85–92% coverage per service`**

`Python 3.12 · FastAPI · RabbitMQ (aio-pika) · PostgreSQL 16 · Pydantic v2 · Docker Compose · pytest · testcontainers · structlog · GitHub Actions`

E-commerce order processing system with **3 microservices** (order, inventory, notification) communicating exclusively through **events** via RabbitMQ, applying Clean Architecture + DDD in each service.

- **Idempotency** in consumers (`processed_events` with `INSERT ... ON CONFLICT DO NOTHING`): a redelivered event does not duplicate business effects.
- **Atomicity anti race-condition** on stock reservation (conditional `UPDATE` + row-level locking + `SAVEPOINT`).
- **Dead-letter queues + retries with backoff** (3 stages) with transient/permanent failure classification.
- **Eventual consistency** via event choreography, no distributed transaction.
- **Observability**: structured JSON logging + **end-to-end correlation ID** (HTTP → events → 3 services) and robust broker reconnection.
- **database-per-service**; **138 tests**, **85–92% coverage** per service — integration against **real PostgreSQL via testcontainers** (SQLite lacks the row-level locking needed to verify concurrency).
- **CI/CD**: GitHub Actions with per-service matrix (lint + mypy + tests).

Repo: github.com/MauriDev94/event-driven-orders

MedBook API — Medical appointment booking with Django + DRF

> **`274 tests · 98.7% branch coverage · 0 failures · strict TDD`**

`Python 3.12 · Django 5.1 · DRF · PostgreSQL 16 · simplejwt · drf-spectacular · pytest · factory-boy · Docker · GitHub Actions · Codecov`

REST API for managing medical appointments, covering the full booking lifecycle: doctors publish schedules, the system generates concrete slots, and patients book them, moving through a **state machine** (pending → confirmed → completed / cancelled / no-show).

- **274 tests · 98.7% branch coverage · 0 failures** — built with **strict TDD** (Red-Green-Refactor) and `--cov-fail-under` enforced in CI.
- **RBAC** with 8 permission classes and object-level role-based access (doctor/patient/admin).
- **Atomic transactions + atomic UPDATE** to prevent *race conditions* when booking concurrent slots.
- ORM optimization: `select_related`/`prefetch_related` (zero N+1), `annotate()`/`Q()` for queries.
- **OpenAPI 3** documentation with drf-spectacular (Swagger UI) and email notifications (django-anymail/Resend).
- **Rate limiting** (AnonRateThrottle) for brute-force protection on login.
- **CI/CD**: GitHub Actions with lint (ruff), tests against real PostgreSQL, and coverage to Codecov.

Repo: github.com/MauriDev94/MedBook · Live (Swagger UI): medbook-oiio.onrender.com/api/docs

API Monolith — Modular monolith with Clean Architecture

> `3 domain modules · JWT/OAuth2/OTP/SSO auth · branch coverage gated at 82%`

`Python · FastAPI · PostgreSQL · SQLAlchemy · Alembic · Pytest · Docker · JWT · OAuth2 · OTP · SSO · GitHub Actions · Render`

REST API built with **FastAPI** applying Clean Architecture, SOLID, and DDD. **3 domain modules** (Auth, Users, Todos), each with its own `domain / application / infrastructure / presentation / di` layers. Auth includes JWT/OAuth2, OTP, password reset, and Google login via SSO.

- Each module defines its **own contracts** (`application/contracts/`) under a strict dependency rule: **the domain has zero framework dependencies** and infrastructure is swappable without touching business logic.
- Cross-cutting middleware (logging, error handling, security); PostgreSQL schemas with **SQLAlchemy** and migrations with **Alembic**.
- **CI/CD** across two jobs: *quality* (ruff, black, mypy) and *tests* against real PostgreSQL with `--cov-branch --cov-fail-under=82`. **Deployed to production on Render.**

Repo: github.com/MauriDev94/Api_monolith · Live (Swagger UI): api-monolith.onrender.com/docs

TECHNICAL SKILLS

- **Languages:** Python, SQL
- **Frameworks:** FastAPI, Django, Django REST Framework (DRF)
- **Messaging / event-driven:** RabbitMQ (aio-pika), idempotency, dead-letter queues, retries with backoff, eventual consistency
- **Databases:** PostgreSQL, MongoDB — schema design and queries, ORM optimization (zero N+1)
- **ORM & migrations:** SQLAlchemy, Alembic, Django ORM
- **Testing:** Pytest, pytest-django, TDD, factory-boy, testcontainers, unit & integration tests, coverage.py, CI quality gates
- **Auth & security:** JWT, OAuth2, OTP, SSO, Google Login, RBAC, rate limiting (throttling)
- **API:** REST, OpenAPI 3 / Swagger (drf-spectacular), endpoint design, versioning
- **CI/CD:** GitHub Actions (lint, tests, coverage, artifacts), Codecov, pre-commit
- **DevOps:** Docker, Git, GitHub, Render (continuous deployment)
- **Observability:** structlog (JSON logging), end-to-end correlation IDs
- **Quality & methodologies:** SOLID, design patterns, Clean Code, ruff, black · Clean Architecture, DDD, TDD, SDD, event-driven microservices, database-per-service, Modular Monolith

LANGUAGES

- **Spanish:** Native
- **English: C2 Proficient — EF SET 71/100** (technical documentation and communication with remote teams)

EDUCATION

Programmer Analyst — Instituto Profesional Santo Tomás · Santiago, Chile · Mar 2021 – Jul 2025